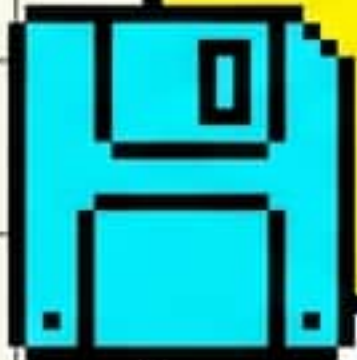
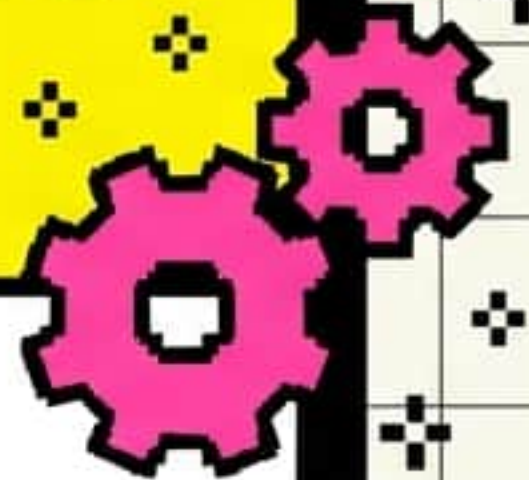


CONTEXT ENGINEERING: 6 KỸ THUẬT THỰC CHIẾN



```
// manual_v2.2 :: system_architecture
```



Bộ tài liệu thiết kế hệ điều hành AI.

AI BẮT ĐẦU MỖI SESSION VỚI MỘT BLANK SLATE.

SYSTEM_WARNING: volatile_memory_detected

Session 1

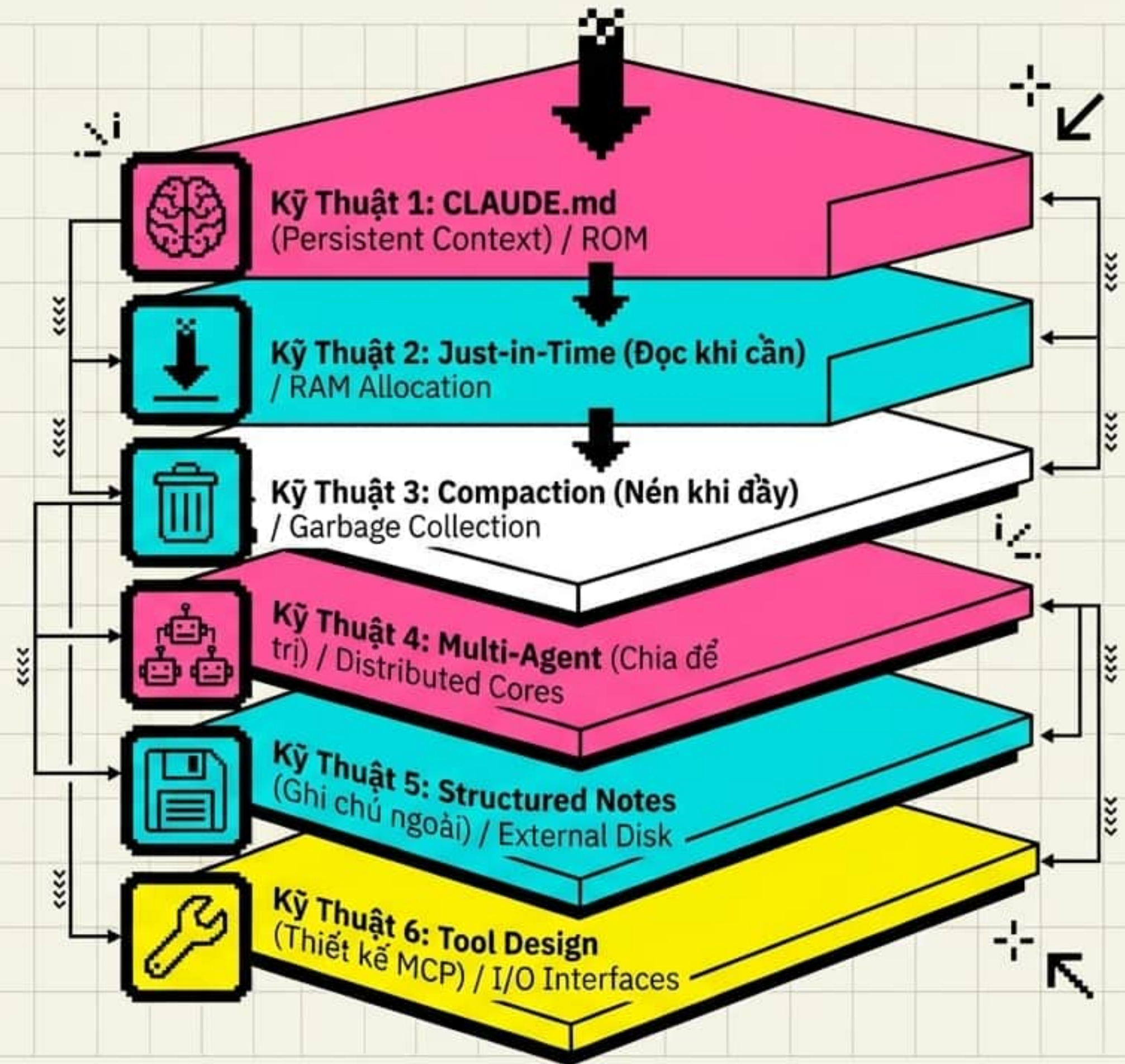
Session 2



Context không phải là phép thuật.
Nó là quản lý bộ nhớ máy tính. **Không cấp phát đúng = Hệ thống crash.**

HỆ THỐNG AI AGENT ENGINE (6 MODULE CỐT LÕI)

// architecture_overview



KỸ THUẬT 1: PERSISTENT CONTEXT

```
// module 01: system_rom
```

CLAUDE.md là bộ nhớ dài hạn. Chứa thông tin agent cần trong MỌI LẦN khởi động.

✓ TỐT (Optimized)

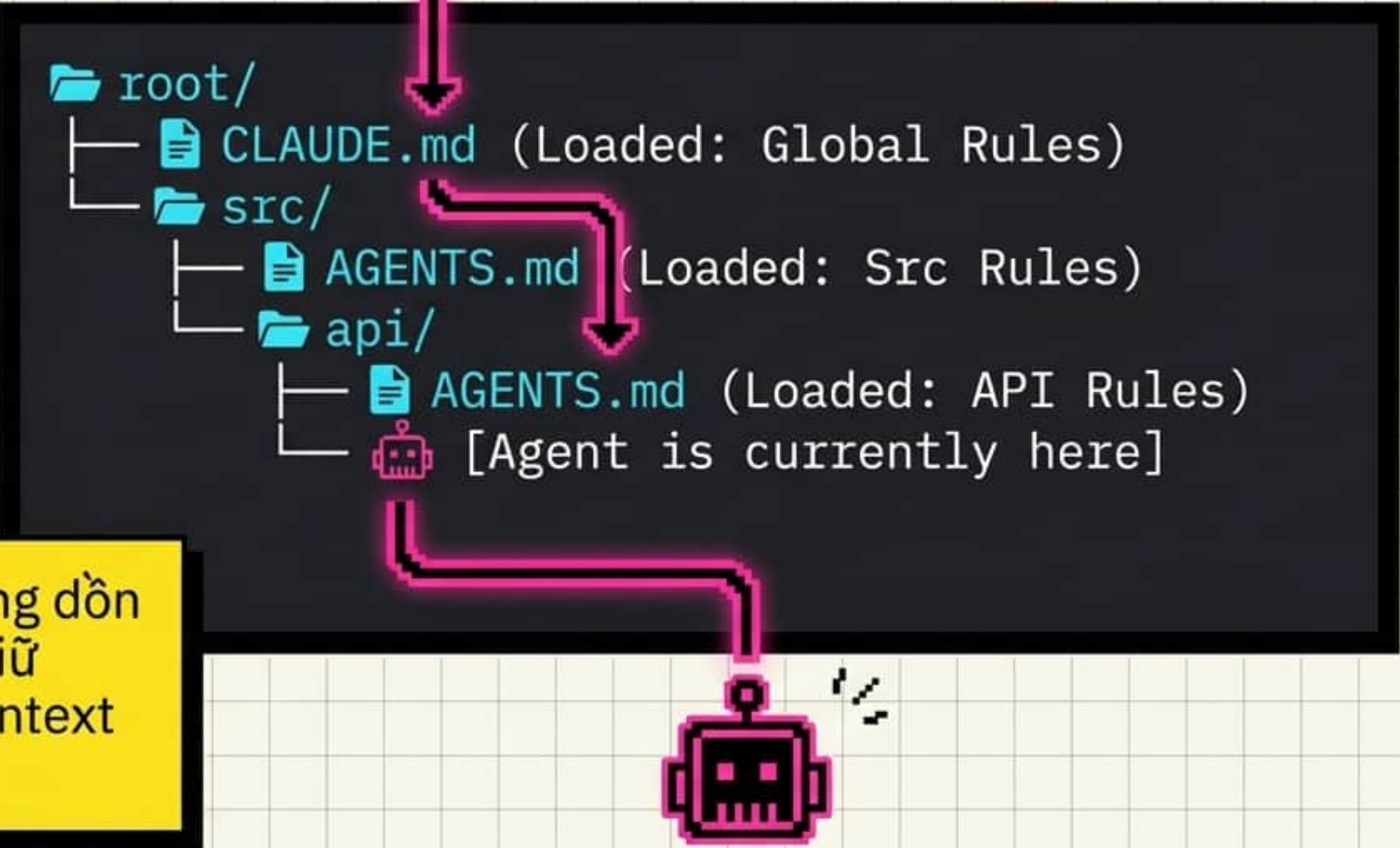
- **Kích thước:** 50-100 dòng.
- **Nội dung:** Thông tin AI không thể tự infer.
- **Chi tiết:** Actionable (dùng X, không dùng Y).
- **Update:** Mỗi khi phát hiện gotcha mới.

✗ XẤU (Bloated)

- **Kích thước:** 300+ dòng ~(~~Token waste!~~).
- **Nội dung:** Copy paste toàn bộ (~~từ README~~).
- **Chi tiết:** Vague (~~hãy viết code tốt~~).
- **Update:** Viết 1 lần (~~rồi quên luôn~~).

MODULE 1.1: HIERARCHICAL CONTEXT

// concept: directory_scoping



The diagram illustrates a hierarchical context structure. It starts with a 'root/' directory containing a 'CLAUDE.md' file (Loaded: Global Rules) and a 'src/' subdirectory. The 'src/' directory contains an 'AGENTS.md' file (Loaded: Src Rules) and an 'api/' subdirectory. The 'api/' directory contains another 'AGENTS.md' file (Loaded: API Rules) and a robot icon representing the current agent. Pink arrows show the flow of context inheritance from the root level down to the 'api/' level. A larger robot icon is positioned at the bottom right of the diagram.

```
root/  
├── CLAUDE.md (Loaded: Global Rules)  
└── src/  
    ├── AGENTS.md (Loaded: Src Rules)  
    └── api/  
        ├── AGENTS.md (Loaded: API Rules)  
        └── [Agent is currently here]
```

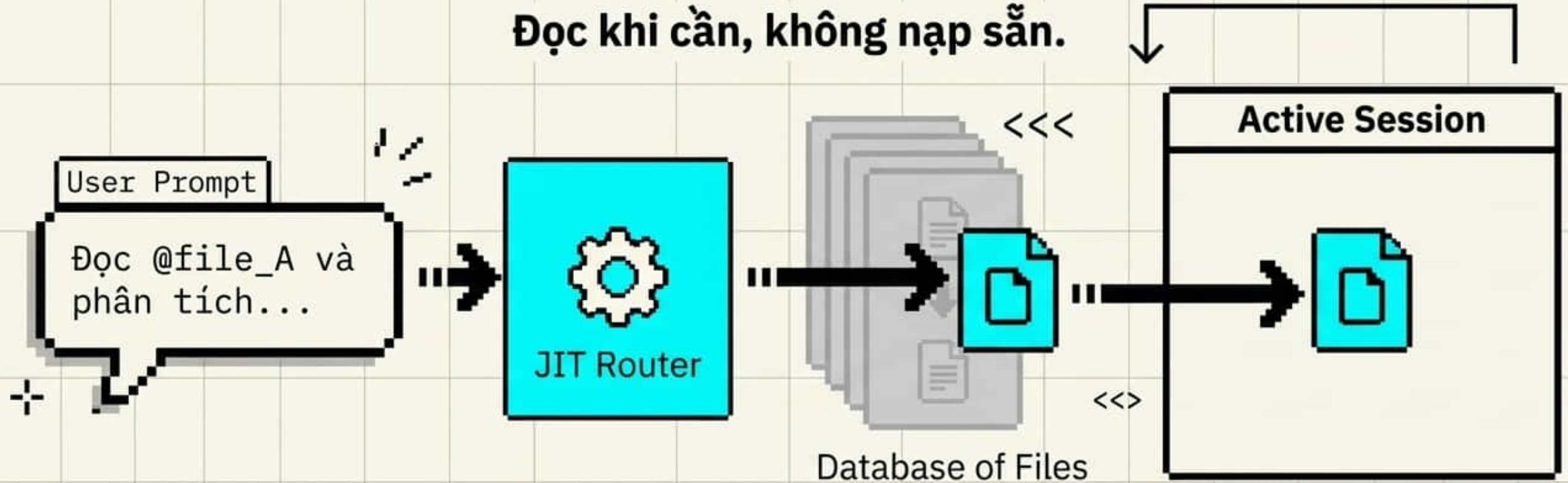
Takeaway Box

Nguyên tắc: Context được cộng dồn (inherit) từ ngoài vào trong. Giữ global context mỏng, local context chi tiết.

KỸ THUẬT 2: JUST-IN-TIME (JIT) LOADING

```
// module 02: dynamic_ram_allocation
```

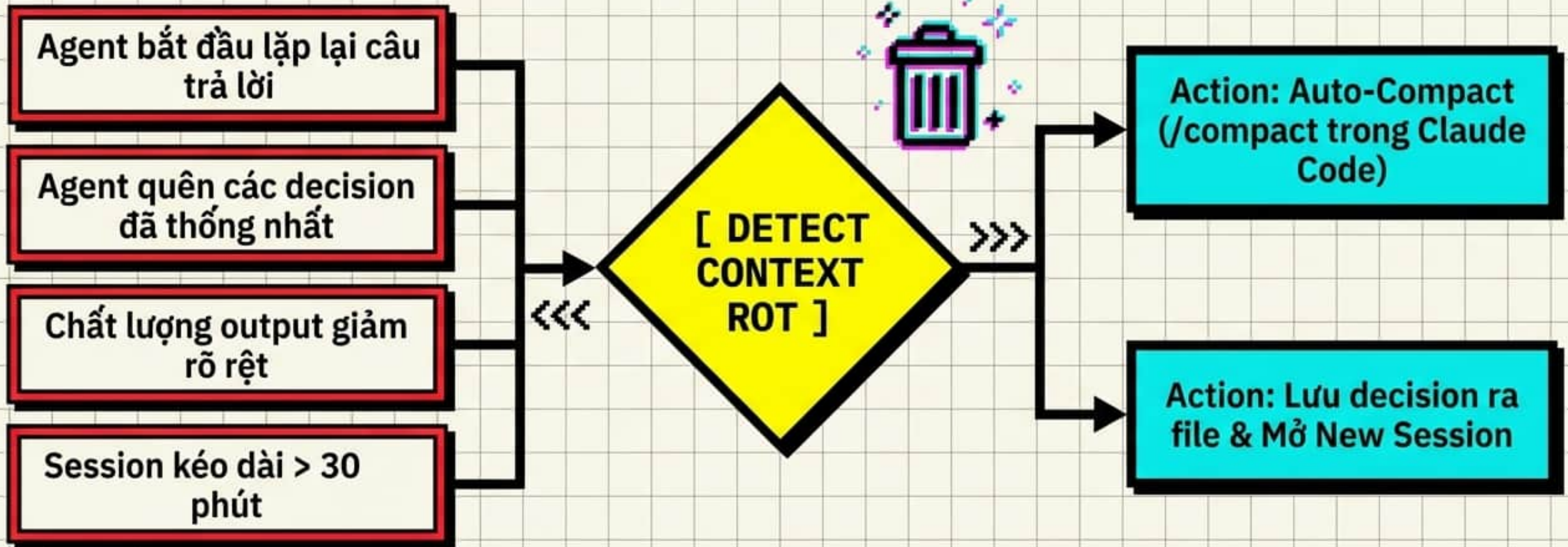
Đọc khi cần, không nạp sẵn.



Thay vì nhồi toàn bộ code base vào CLAUDE.md, hãy dùng JIT loading trong prompt để kéo chính xác file cần thiết.

KỸ THUẬT 3: CONTEXT COMPACTION

```
// module 03: garbage_collection
```



KỸ THUẬT 4: MULTI-AGENT ARCHITECTURE

// module 04: distributed_processing



Chia để trị. Dùng cho các tasks phức tạp và can thiệp nhiều files.
Một agent ôm đồm = Tràn context & Hallucination.

KỸ THUẬT 5: STRUCTURED NOTES

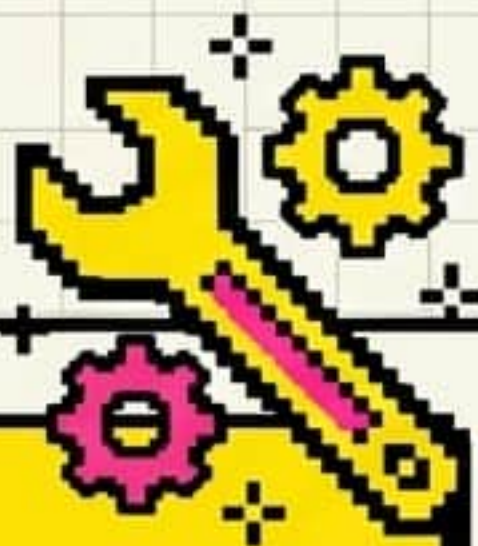
```
// module 05: external_state_drive
```



- Hệ thống dựa trên file (File-based State).
- Lưu lại mọi milestone và quyết định kiến trúc.
- Giải phóng RAM: Agent không cần phải nhớ, chỉ cần đọc lại khi cần.

KỸ THUẬT 6: THIẾT KẾ TOOLS (MCP)

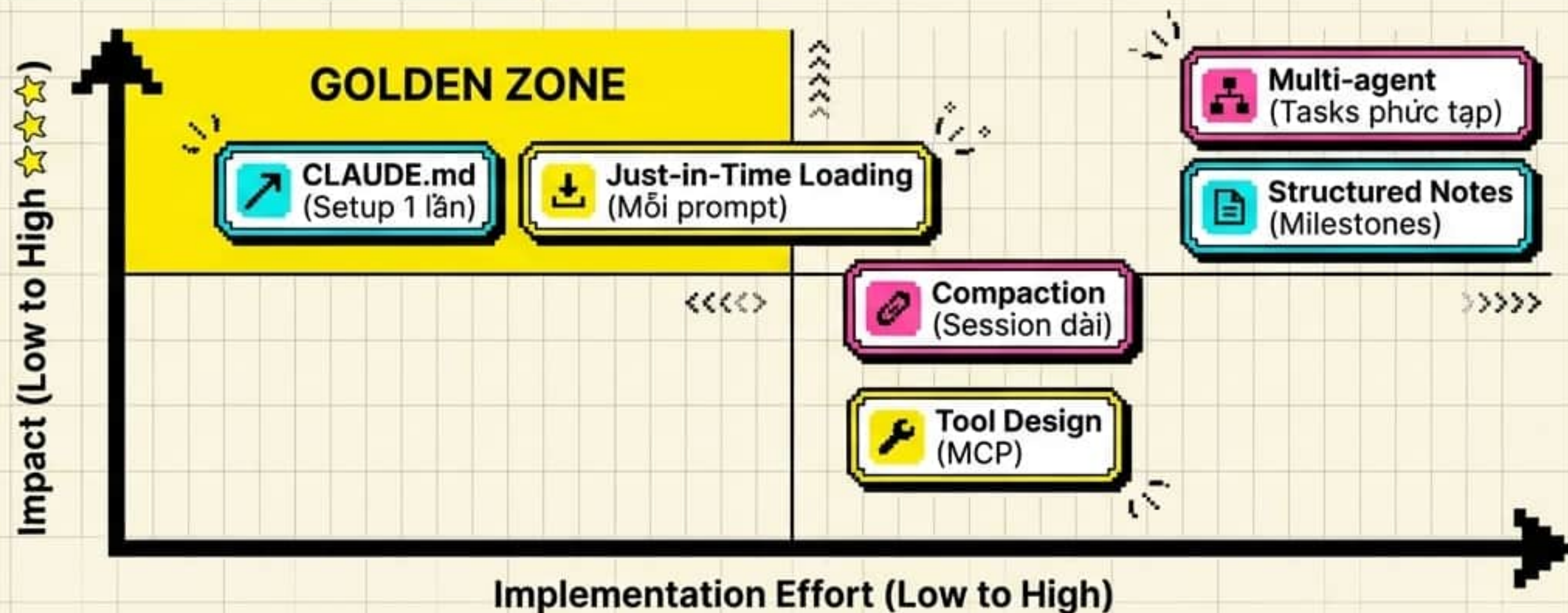
// module 06 : optimized_io_interfaces



Quy Tắc	✅ Tối Ưu (Lean)	❌ Sai Lầm (Bloat)
Mô tả	Query PostgreSQL database	Database stuff
Overlap	query_db + list_tables	db_read + db_query + get_data
Token Efficiency	Return chỉ fields cần thiết	Return toàn bộ raw response JSON
Parameters	get_user(id: int)	get_data(params: dict)
Error Messages	User 123 not found in users table	Error occurred

MA TRẬN QUYẾT ĐỊNH CONTEXT

// synthesis: effort_vs_impact



Bắt đầu với **CLAUDE.md** và **JIT**. Đây là hai kỹ thuật tốn ít công sức nhất nhưng mang lại hiệu quả cao nhất để kiểm soát AI.

- > SYSTEM INITIATING BOOT SEQUENCE...
- > LOADING HOMEWORK MODULES...

[ACTION REQUIRED]

- > EXECUTE LAB_3: Áp dụng kỹ thuật CLAUDE.md vào project hiện tại.
- > EXECUTE LAB_4: Áp dụng JIT loading và đối chứng kết quả output.

[DISCUSSION_PROMPT]

- > Team của bạn đã cần Multi-Agent chưa? Hay CLAUDE.md + JIT là đủ?
- > NEXT_FILE_TO_LOAD: Lesson_2.3_Anti_Patterns.exe
- > _

